COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	July 16, 2026	**DATE SUBMITTED:**	July 21, 2026
NAME:	Dela Cruz, Jared Isaiah P.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 2.1 **Arrays, Pointers and Dynamic Allocation**

1. PROCEDURE OUTPUT

ILOA: Implement static and dynamic memory allocation

SCREENSHOT

```cpp
#include <iostream>
#include <string>

class Student{
    private:
        std::string studentName;
        int studentAge;
    public:
        //constructor
        Student(std::string newName = "John Doe", int newAge = 18){
            studentName = newName;
            studentAge = newAge;
            std::cout << "Constructor Called." << std::endl;
        };

        //Deconstructor
        ~Student(){
            std::cout << "Destructor Called." << std::endl;
        }

        //Copy Constructor
        Student(const Student &copyStudent){
            std::cout << "Copy Constructor Called." << std::endl;
            studentName = copyStudent.studentName;
            studentAge = copyStudent.studentAge;
        }

        //Display Attributes
        void printDetails(){
            std::cout << this->studentName << "  " << this->studentAge << std::endl;
    }
};

int main(){
    Student student1("Redbull", 21);
    Student student2(student1);
    Student student3;
    student3 = student2;

    return 0;
}
```

OBSERVATION

Referring to the code visualizer, I noticed that the code starts with int main() and immediately proceeds to the constructor, which sets Redbull and 21 as the student name and age respectively. After this, the copy constructor was called, but it failed to copy the student name but it copies the age, which is 21. Next, it runs the student3 function to

	declare John Doe and 18 as its studentName and studentAge. The Constructor is run once again to point the addresses on student2 and student3. Afterwards, the deconstructor is called thrice to destroy the contents.

Table 2-1: Initial Driver Program

SCREENSHOT

```
#include <iostream>
#include <string>

class Student{
    private:
        std::string studentName;
        int studentAge;
    public:
        //constructor
        Student(std::string newName = "John Doe", int newAge = 18){
            studentName = newName;
            studentAge = newAge;
            std::cout << "Constructor Called." << std::endl;
        };

        //Deconstructor
        ~Student(){
            std::cout << "Destructor Called." << std::endl;
        }

        //Copy Constructor
        Student(const Student &copyStudent){
            std::cout << "Copy Constructor Called." << std::endl;
            studentName = copyStudent.studentName;
            studentAge = copyStudent.studentAge;
        }

        //Display Attributes
        void printDetails(){
            std::cout << this->studentName << "   " << this->studentAge << std::endl;
    }
};

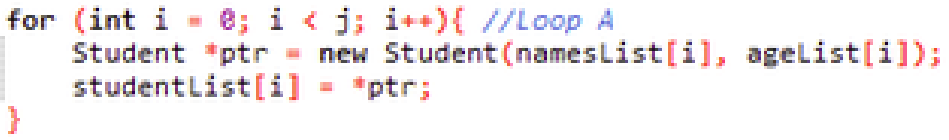
int main(){
    const size_t j = 5;

    Student studentList[j] = {};
    std::string namesList[j] = {"Dylan", "Sophia", "Jared", "Kurt", "Brodie"};
    int ageList[j] = {21, 22, 20, 23, 25};

    return 0;
}
```

OBSERVATION	Because of the new driver function, despite similarities with the first program, it now declares a specific array size of 5. This also affects the student list, names list, and age list, all of which will now contain 5 slots. For the studentsList, it will all be filled by John Doe and 18 for the name and age respectively, which is why the constructor was called 5 times. For the final run, it then initializes the namesList and ageList. After which the Destructor is called 5 times as well, deleting the contents.

Table 2-2: Modified Driver Program with Student Lists

LOOP A	
OBSERVATION	Because of the loop function, the visualizer fails to represent what was going on since it exceeds 55 tasks available for simulation. As I sought ChatGPT for help to explain what was going on, apparently this program contains a memory leak since we allocated memory with “new” without calling “delete. The same thing happened when I tried decreasing the number of allocated arrays. For the program, it called the constructor ten times and the destructor 5 times. At the end, it destroyed the string list but the student list remained the same, containing Dylan, Sophia, and Jared with all their ages. In addition, since the program used object assignment, copy construction was never called.
LOOP B	
OBSERVATION	This program is almost similar to Loop A but it instead uses printDetails instead of making a *ptr with new. It shows that each element is initialized independently using the default constructor and destroyed automatically when the execution ends.Since the student data is never updated with the values from namesList and ageList, every object retains its default values, which are John Doe and 18, making those two arrays redundant in the current implementation.

OUTPUT

```cpp
#include <iostream>
#include <string>

class Student{
    private:
        std::string studentName;
        int studentAge;
    public:
        //constructor
        Student(std::string newName = "John Doe", int newAge = 18){
            studentName = newName;
            studentAge = newAge;
            std::cout << "Constructor Called." << std::endl;
        };

        //Deconstructor
        ~Student(){
            std::cout << "Destructor Called." << std::endl;
        }

        //Copy Constructor
        Student(const Student &copyStudent){
            std::cout << "Copy Constructor Called." << std::endl;
            studentName = copyStudent.studentName;
            studentAge = copyStudent.studentAge;
        }

        //Display Attributes
        void printDetails(){
            std::cout << this->studentName << "  " << this->studentAge << std::endl;
    }
};

int main(){
    const size_t j = 5;

    Student studentList[j] = {};
    std::string namesList[j] = {"Dylan", "Sophia", "Jared", "Kurt", "Brodie"};
    int ageList[j] = {21, 22, 20, 23, 25};

    for (int i = 0; i < j; i++){ //Loop A
        Student *ptr = new Student(namesList[i], ageList[i]);
        studentList[i] = *ptr;
    }

    for(int i = 0; i < j; i++){ //Loop B
        studentList[i].printDetails();
    }

    return 0;
}
```

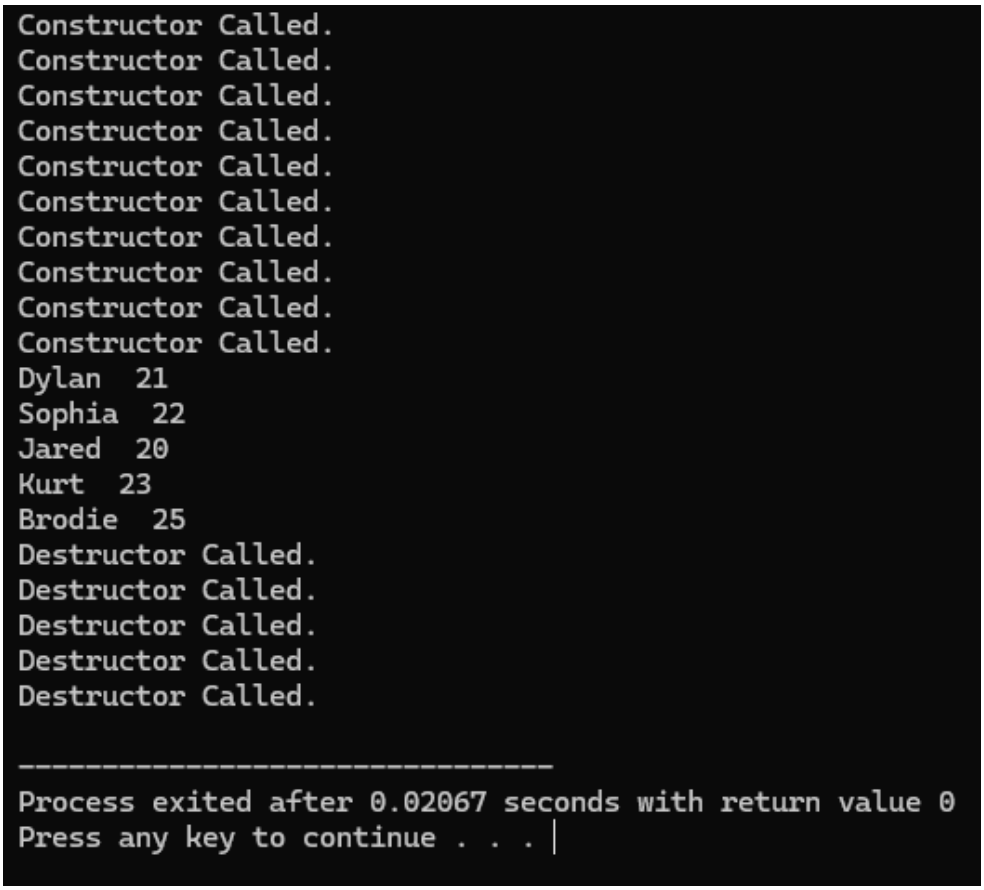
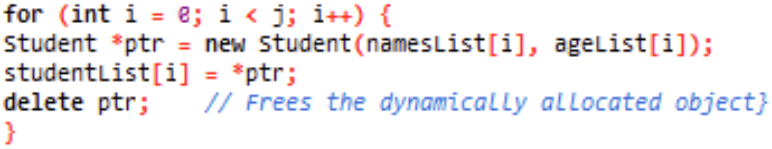
	
OBSERVATION	With both the loops combined, it successfully showcases the difference between creating objects on the stack (studentList) and on the heap (new Student(...). Instead of John Doe and 18, it is then replaced by the declared values on the string. However, upon further consultation, there's still a memory leak apparently, and adding a delete function would be able to improve the program for efficiency.

Table 2-3: Final Driver Program

MODIFICATIONS	 for (int i = 0; i < j; i++) {
	Student *ptr = new Student(namesList[i], ageList[i]);
	studentList[i] = *ptr;
	delete ptr; // Frees the dynamically allocated object}
	}

OBSERVATION	By adding a delete ptr function in Loop A, it then deletes each dynamically created object, preventing further memory leaks.

Table 2-4: Modifications/Corrections Necessary

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve programming problems using dynamic memory allocation, arrays and pointers

Jenna's Grocery List		
Apple	PHP 10	x7
Banana	PHP 10	x8
Broccoli	PHP 60	x12
Lettuce	PHP 50	x10

Jenna wants to buy the following fruits and vegetables for her daily consumption. However, she needs to distinguish between fruit and vegetable, as well as calculate the sum of prices that she has to pay in total.

Problem 1: Create a class for the fruit and the vegetable classes. Each class must have a constructor, deconstructor, copy constructor and copy assignment operator. They must also have all relevant attributes (such as name, price and quantity) and functions (such as calculate sum) as presented in the problem description above.

Problem 2: Create an array GroceryList in the driver code that will contain all items in Jenna's Grocery List. You must then access each saved instance and display all details about the items.

Problem 3: Create a function TotalSum that will calculate the sum of all objects listed in Jenna's Grocery List.

Problem 4: Delete the Lettuce from Jenna's GroceryList list and de-allocate the memory assigned.

Problem 5: Discuss the

ANSWERS:

```cpp
#include <iostream>
#include <string>

// Attribute: Fruit
class Fruit {
private:
    std::string name;
    double price;
    int quantity;

public:
    // Constructor
    Fruit(std::string newName = "Fruit", double newPrice = 0.0, int newQty = 0) {
        name = newName;
        price = newPrice;
        quantity = newQty;
        std::cout << "Constructor: Fruit Called for " << name << "." << std::endl;
    }

    // Destructor
    virtual ~Fruit() {
        std::cout << "Destructor: Fruit Called for " << name << "." << std::endl;
    }

    // Copy Constructor
    Fruit(const Fruit &copyFruit) {
        std::cout << "Copy Constructor: Fruit Called." << std::endl;
        name = copyFruit.name;
        price = copyFruit.price;
        quantity = copyFruit.quantity;
    }

    // Copy Assignment
    Fruit& operator=(const Fruit &other) {
        std::cout << "Copy Assignment: Fruit Called." << std::endl;
        if (this != &other) {
            name = other.name;
            price = other.price;
            quantity = other.quantity;
        }
        return *this;
    }

    // Member Functions
    double calculateSum() const {
        return price * quantity;
    }

    void printDetails() const {
        std::cout << name << " - PHP each: " << price << " - Pcs" << quantity << " - Subtotal: PHP " << calculateSum() << std::endl;
    }

```

```cpp
    std::string getName() const { return name; }
    double getPrice() const { return price; }
    int getQuantity() const { return quantity; }
};

// Attribute: Vegetable
class Vegetable {
private:
    std::string name;
    double price;
    int quantity;

public:
    // Constructor
    Vegetable(std::string newName = "Vegetable", double newPrice = 0.0, int newQty = 0) {
        name = newName;
        price = newPrice;
        quantity = newQty;
        std::cout << "Constructor: Vegetable Called for " << name << "." << std::endl;
    }

    // Destructor
    virtual ~Vegetable() {
        std::cout << "Destructor: Vegetable Called for " << name << "." << std::endl;
    }

    // Copy Constructor
    Vegetable(const Vegetable &copyVeg) {
        std::cout << "Copy Constructor: Vegetable Called." << std::endl;
        name = copyVeg.name;
        price = copyVeg.price;
        quantity = copyVeg.quantity;
    }

    // Copy Assignment
    Vegetable& operator=(const Vegetable &other) {
        std::cout << "Copy Assignment: Vegetable Called." << std::endl;
        if (this != &other) {
            name = other.name;
            price = other.price;
            quantity = other.quantity;
        }
        return *this;
    }

    // Member Functions
    double calculateSum() const {
        return price * quantity;
    }

    void printDetails() const {
        std::cout << name << " - PHP each: " << price << " - Pcs" << quantity << " - Subtotal: PHP " << calculateSum() << std::endl;
```

```
107         std::string getName() const { return name; }
108         double getPrice() const { return price; }
109         int getQuantity() const { return quantity; }
110     };
111
112     // Polymorphic Wrappers
113     class GroceryItem {
114     private:
115         Fruit* fruitPtr;
116         Vegetable* vegPtr;
117
118     public:
119         // Dynamic Constructors
120         GroceryItem() : fruitPtr(NULL), vegPtr(NULL) {}
121         GroceryItem(const Fruit& f) : fruitPtr(new Fruit(f)), vegPtr(NULL) {}
122         GroceryItem(const Vegetable& v) : fruitPtr(NULL), vegPtr(new Vegetable(v)) {}
123
124         // Destructor
125         ~GroceryItem() {
126             delete fruitPtr;
127             delete vegPtr;
128         }
129
130         // Copy Constructor
131         GroceryItem(const GroceryItem& other) {
132             fruitPtr = other.fruitPtr ? new Fruit(*other.fruitPtr) : NULL;
133             vegPtr = other.vegPtr ? new Vegetable(*other.vegPtr) : NULL;
134         }
135
136         // Copy Assignment
137         GroceryItem& operator=(const GroceryItem& other) {
138             if (this != &other) {
139                 delete fruitPtr;
140                 delete vegPtr;
141                 fruitPtr = other.fruitPtr ? new Fruit(*other.fruitPtr) : NULL;
142                 vegPtr = other.vegPtr ? new Vegetable(*other.vegPtr) : NULL;
143             }
144             return *this;
145         }
146
147         double getItemSum() const {
148             if (fruitPtr) return fruitPtr->calculateSum();
149             if (vegPtr) return vegPtr->calculateSum();
150             return 0.0;
151         }
152
153         void print() const {
154             if (fruitPtr) fruitPtr->printDetails();
155             else if (vegPtr) vegPtr->printDetails();
156         }
157
158         std::string getName() const {
159             if (fruitPtr) return fruitPtr->getName();
```

```cpp
        if (vegPtr) return vegPtr->getName();
        return "";
    }
};

// TotalSum Function
double TotalSum(GroceryItem* list, int size) {
    double total = 0.0;
    for (int i = 0; i < size; i++) {
        total += list[i].getItemSum();
    }
    return total;
}

int main() {
    int listSize = 4;

    // Dynamic Array Allocation
    GroceryItem* GroceryList = new GroceryItem[listSize];

    // Dynamic Assignment using Copy Assignment
    GroceryList[0] = GroceryItem(Fruit("Apple", 10.0, 7));
    GroceryList[1] = GroceryItem(Fruit("Banana", 10.0, 8));
    GroceryList[2] = GroceryItem(Vegetable("Broccoli", 60.0, 12));
    GroceryList[3] = GroceryItem(Vegetable("Lettuce", 50.0, 10));

    std::cout << "\nGrocery Details" << std::endl;
    for (int i = 0; i < listSize; i++) {
        GroceryList[i].print();
    }

    // Calling TotalSum
    std::cout << "Total Price: PHP " << TotalSum(GroceryList, listSize) << std::endl;

    int newSize = listSize - 1;
    GroceryItem* UpdatedGroceryList = new GroceryItem[newSize];

    int newIdx = 0;
    for (int i = 0; i < listSize; i++) {
        if (GroceryList[i].getName() != "Lettuce") {
            UpdatedGroceryList[newIdx] = GroceryList[i];
            newIdx++;
        } else {
            std::cout << "Removing Lettuce from Grocery List" << std::endl;
        }
    }

    // Deallocation
    delete[] GroceryList;
    GroceryList = UpdatedGroceryList;
    listSize = newSize;

    std::cout << "\nUpdated Grocery List" << std::endl;
```

```
213         for (int i = 0; i < listSize; i++) {
214             GroceryList[i].print();
215         }
216
217         std::cout << "\nUpdated Total Price: PHP " << TotalSum(GroceryList, listSize) << std::endl;
218
219         delete[] GroceryList;
220
221         return 0;
222     }
```

OBSERVATIONS: Using the main procedure as a basis, I just tried to mimic the parts that I deem may be useful for the supplementary activity. I used the main focus of our topics, such as Heap memory by utilizing the delete and new functions, the rule of three (Constructor, Deconstructor, Copy Constructor and Assignment), and pointers and dynamic arrays for allocation and deallocation.

3. CONCLUSION

- For me, this lab activity was helpful in reinforcing the basic concepts of programming such as object lifecycles, dynamic memory allocations, pointers and the rule of three in C++ t. Examining the procedures involved in our activity helped me understand how objects are created, how deep copy is achieved, and how heap deallocation is done using delete pointers ("Dylan", "Sophia", "Jared", "Kurt", "Brodie"). However, applying the above knowledge to the additional activity revealed the necessity of using polymorphism and dynamic arrays in dealing with mixed instances of Fruit and Vegetables, where I had go and do some research about it to help me carry out the supplementary tasks to create dynamic arrays, calculate subtotals using TotalSum(), and allocate new heap memory upon deletion of an item, "Lettuce", as per the instructions.Overall, I did the activity well by successfully applying memory safety concepts in real programming but my weakness lies in being more independent when it comes to program making. I still need some kind of assistance in order to help me understand things better.

4. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used:
ChatGPT
Gemini

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **AI assistance is used for code debugging and explanation.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **AI was used to confirm if the given code will return the right output and to explain what the process was within the program for further understanding.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

COMPUTER ENGINEERING

QUEZON CITY

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.